

ScholarPath — Class Diagrams (UML)

Domain model and architecture (Sommerville notation)

ScholarPath Graduation Project

June 30, 2026

Abstract

This report presents the ScholarPath domain model as UML class diagrams in the style of Sommerville (*Software Engineering*, 9th ed.). To keep every figure legible, the model is split by subject area; a separate diagram shows the base type hierarchy, and a final “ports & adapters” diagram captures the Clean Architecture seam between the application and infrastructure layers. Classes, attributes and key operations are taken directly from `ScholarPath.Domain/Entities` and the application interfaces.

Contents

1	Notation	1
2	Base entity types	1
3	Identity & Profile	2
4	Scholarships, Applications & Documents	3
5	Booking, Payments & Ratings	4
6	Community & Chat	4
7	Resources Hub & Notifications	5
8	AI, Knowledge, Platform & Cross-cutting	5
9	Architecture — ports & adapters	6

1 Notation

- A **class** is a box with name / attributes / operations. `«abstract»` marks an abstract class; `«interface»` an interface.
- **Generalization** (inheritance) is a hollow triangle pointing at the parent (`<|`, drawn `Base <| - Derived`).
- **Realization** (a class implements an interface) is a dashed hollow triangle (`Interface <|.. Class`).
- **Composition** (filled diamond) means the part cannot exist without the whole; **aggregation** (hollow diamond) means it can.
- **Associations** carry role names and multiplicities (1, 0..1, *).

2 Base entity types

Almost every persistent class inherits `AuditableEntity` (itself a `BaseEntity`); classes that support soft delete also implement `ISoftDeletable`. To avoid drawing this inheritance on every figure, the hierarchy is shown once here and implied thereafter.

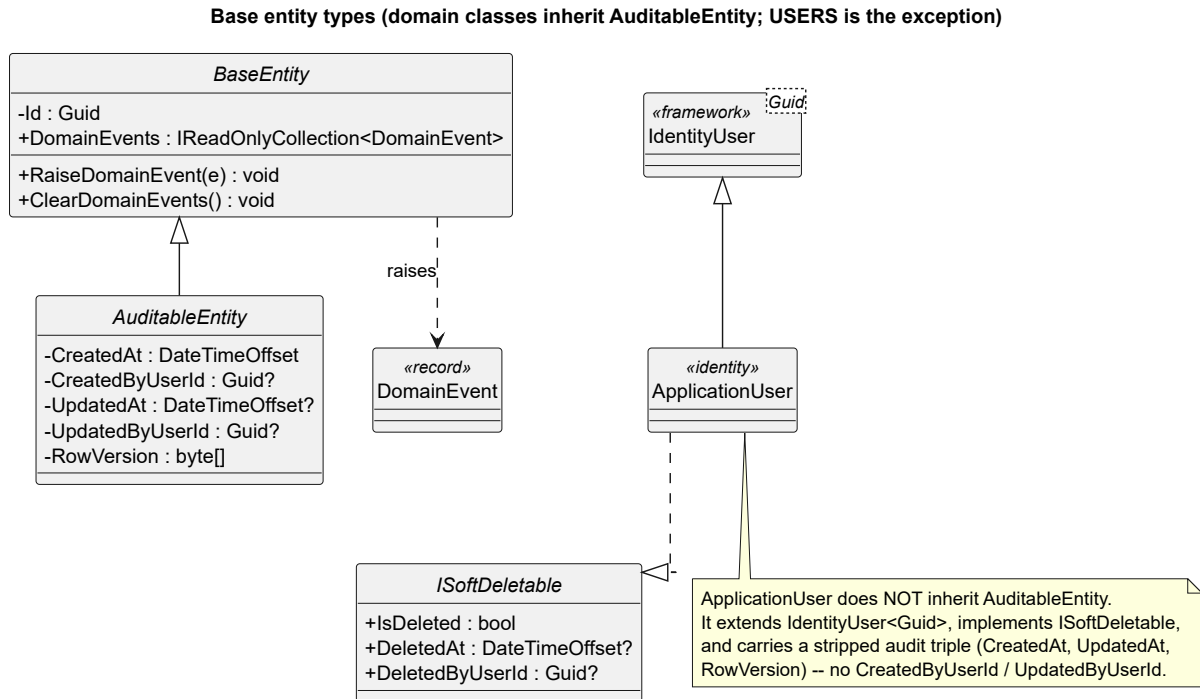


Figure 1: Base entity types and the domain-event mechanism.

3 Identity & Profile

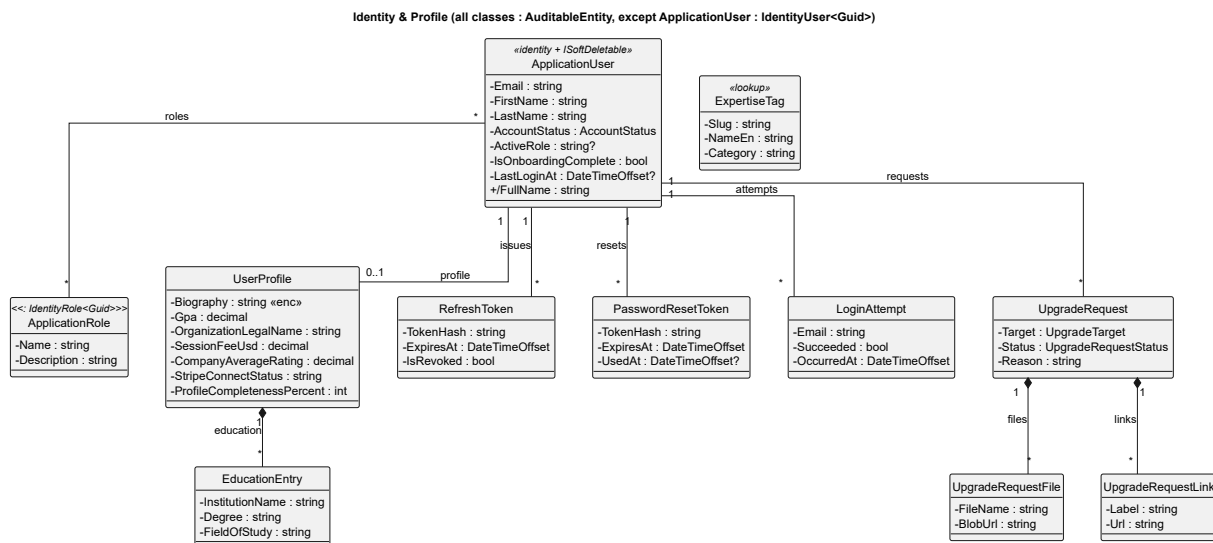


Figure 2: Identity & Profile classes.

Inheritance note. Unlike every other persistent class, `ApplicationUser` does *not* inherit `AuditableEntity`: it extends `IdentityUser<Guid>` and implements `ISoftDeletable`, and therefore carries only a

stripped audit triple (`CreatedAt`, `UpdatedAt`, `RowVersion`) — it has no `CreatedByUserId` / `UpdatedByUserId`. `ApplicationRole` extends `IdentityRole<Guid>`. `ApplicationUser` owns a single `UserProfile` (which composes `EducationEntry` rows), holds many `ApplicationRoles` via the `UserRoles` junction, issues `RefreshToken` / `PasswordResetToken` rows, and may raise `UpgradeRequests` (which compose `UpgradeRequestFile` / `UpgradeRequestLink`). `ExpertiseTag` is a lookup table. `/FullName` is a derived property.

4 Scholarships, Applications & Documents

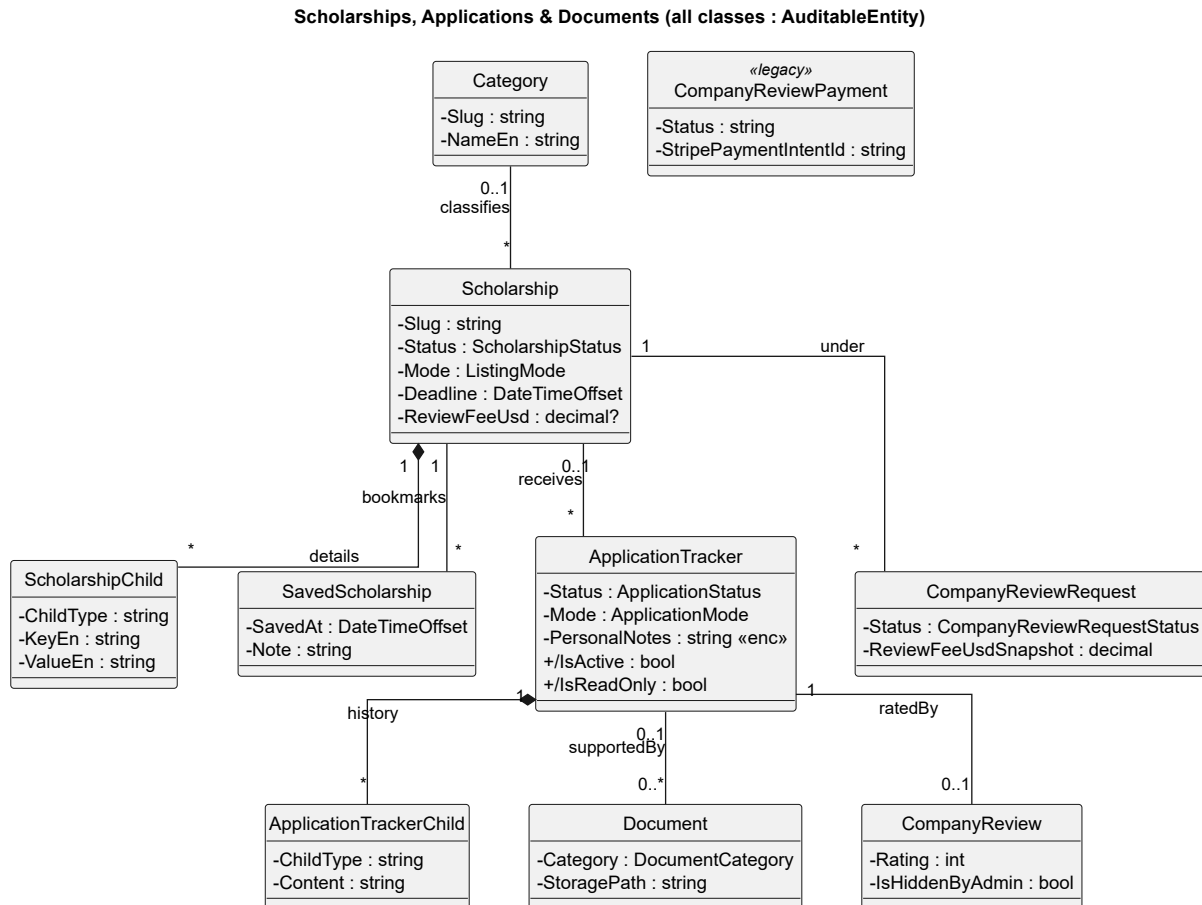


Figure 3: Scholarships, Applications & Documents classes.

A `Scholarship` (optionally classified by a `Category`) receives many `ApplicationTrackers`. An application can be backed by a `Document` and finalised by exactly one `CompanyReview`. `CompanyReviewRequest` models the paid-support engagement. `ApplicationTracker.IsActive` / `IsReadOnly` are computed guards that encode the application state machine.

5 Booking, Payments & Ratings

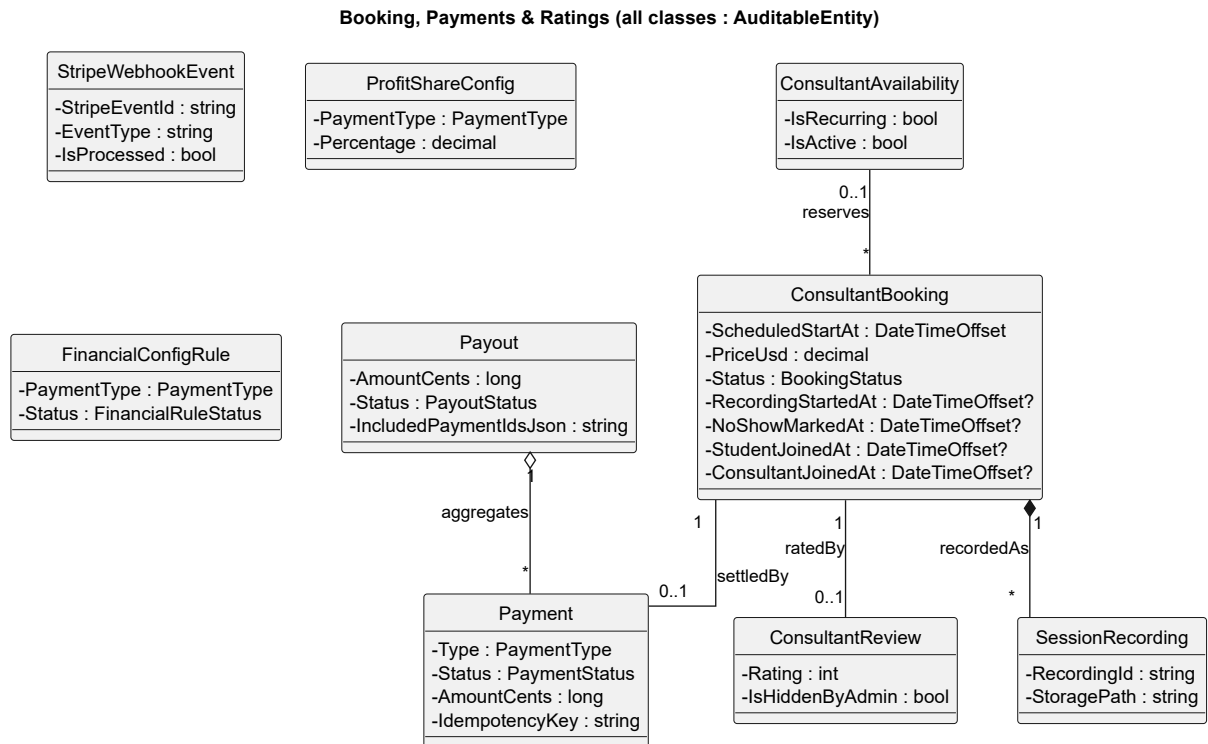


Figure 4: Booking, Payments & Ratings classes.

A `ConsultantBooking` reserves a `ConsultantAvailability`, is settled by one `Payment`, may be rated by one `ConsultantReview`, and is composed of `SessionRecordings`. A `Payout` aggregates many `Payments`. Monetary amounts are integer *cents*.

6 Community & Chat

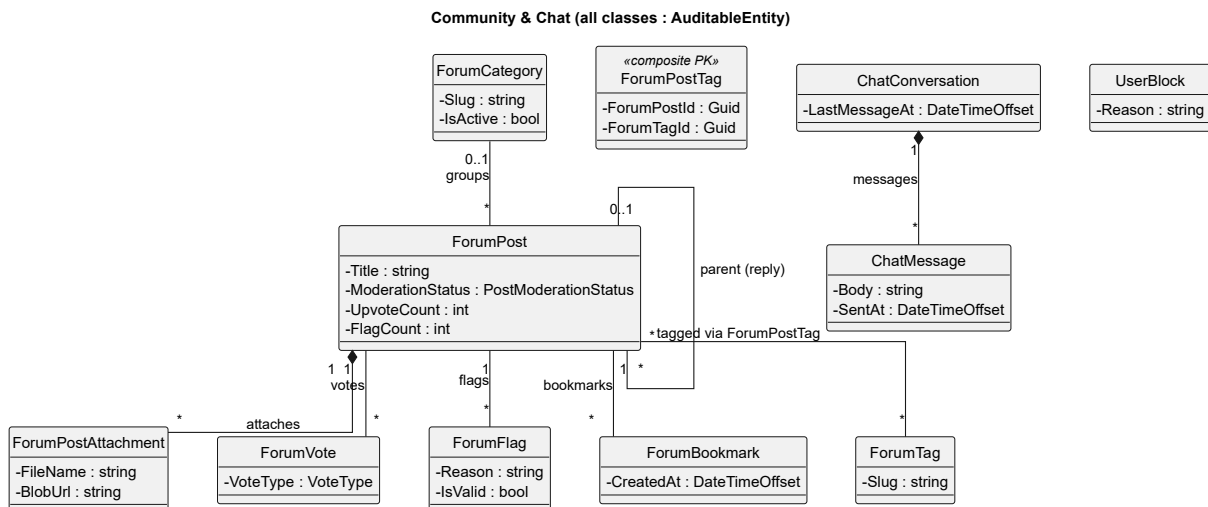


Figure 5: Community & Chat classes.

`ForumPost` is self-referencing (threaded replies), composes `ForumPostAttachments`, receives `ForumVote/ForumFlag`, composes `ForumBookmark` rows, and is tagged many-to-many with `ForumTag` via `ForumPostTag`. `ChatConversation` composes `ChatMessages`; `UserBlock` records a directed block between two users.

7 Resources Hub & Notifications

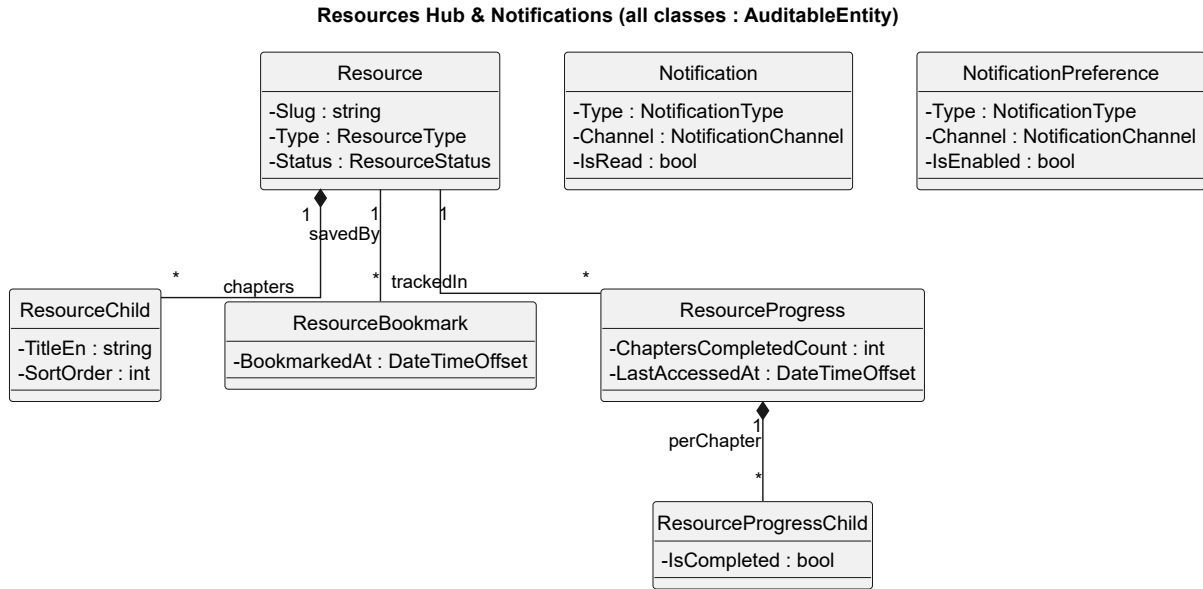


Figure 6: Resources Hub & Notifications classes.

A `Resource` is split into `ResourceChild` chapters and is saved/tracked through `ResourceBookmark` and `ResourceProgress` (itself composing `ResourceProgressChild`). Notification delivery preferences live in `NotificationPreference`.

8 AI, Knowledge, Platform & Cross-cutting

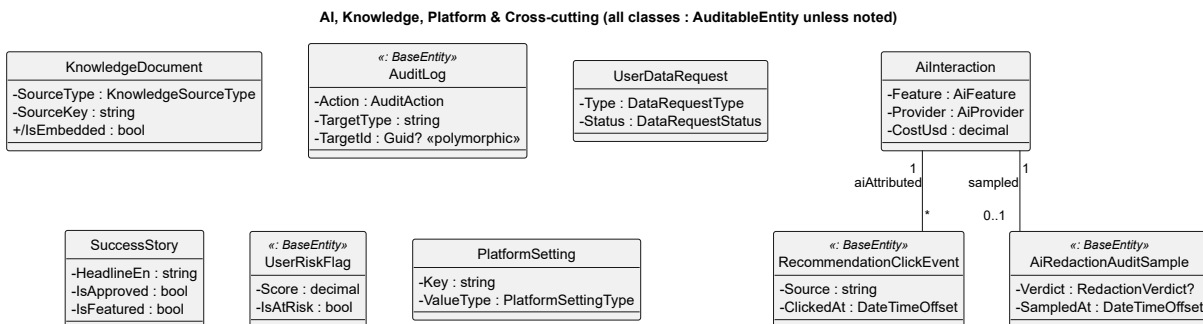


Figure 7: AI, Knowledge, Platform & Cross-cutting classes.

`AiInteraction` may be sampled by an `AiRedactionAuditSample` and attributed to many `RecommendationClickEvent`. `KnowledgeDocument` backs the RAG pipeline. `AuditLog`, `UserDataRequest`, `UserRiskFlag`, `SuccessStory` and `PlatformSetting` are the cross-cutting tables; those marked : `BaseEntity` are intentionally *not* auditable (append-only / analytics rows).

9 Architecture — ports & adapters

The application layer defines **ports** (interfaces); the infrastructure layer provides **adapters** (implementations), chosen at start-up by dependency injection / configuration. This seam is why the platform can swap a development stub for a real Azure service without touching application code.

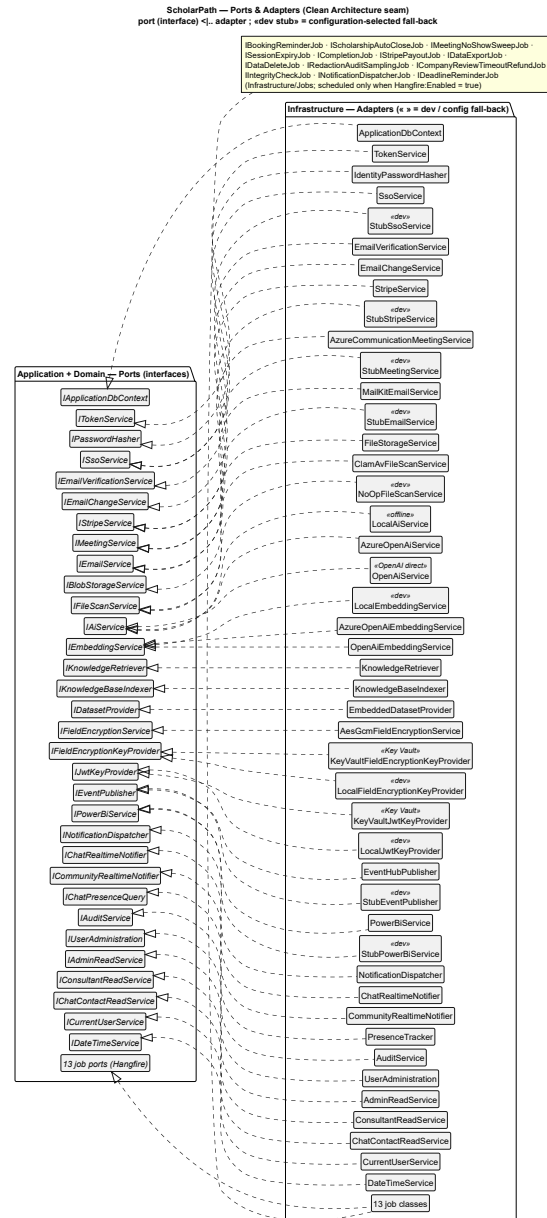


Figure 8: Ports (interfaces) and their infrastructure adapters.

Port (interface)	Production adapter	External system
IApplicationDbContext	ApplicationDbContext (EF Core)	Azure SQL
ITokenService	TokenService (JWT signer)	—
IPasswordHasher	IdentityPasswordHasher	—
ISsoService	SsoService / StubSsoService	Google + Microsoft OAuth
IEmailVerificationService	EmailVerificationService	—
IEmailChangeService	EmailChangeService	—

Port (interface)	Production adapter	External system
IStripeService	StripeService / <i>StubStripeService</i>	Stripe
IMeetingService	AzureCommunicationMeetingService / <i>StubMeetingService</i>	Azure Comm. Svcs
IEmailService	MailKitEmailService / <i>StubEmailService</i>	SMTP
IBlobStorageService	FileStorageService	Azure Blob / local
IFileScanService	ClamAvFileScanService / <i>NoOpFileScanService</i>	ClamAV
IAiService	AzureOpenAiService / OpenAiService / <i>LocalAiService</i>	Azure OpenAI or OpenAI direct
IEmbeddingService	AzureOpenAiEmbeddingService / OpenAiEmbeddingService / <i>LocalEmbeddingService</i>	Azure OpenAI or OpenAI direct
IKnowledgeRetriever	KnowledgeRetriever	(RAG read)
IKnowledgeBaseIndexer	KnowledgeBaseIndexer	(RAG write)
IDatasetProvider	EmbeddedDatasetProvider	(bundled corpora)
IFieldEncryptionService	AesGcmFieldEncryptionService	—
IFieldEncryptionKeyProvider	KeyVault... / <i>Local...</i>	Azure Key Vault
IJwtKeyProvider	KeyVaultJwtKeyProvider / <i>Local...</i>	Azure Key Vault
IEventPublisher	EventHubPublisher / <i>StubEventPublisher</i>	Azure Event Hub
IPowerBiService	PowerBiService / <i>StubPowerBiService</i>	Power BI
INotificationDispatcher	NotificationDispatcher	in-app + email + SignalR
IChatRealtimeNotifier	ChatRealtimeNotifier	SignalR ChatHub
ICommunityRealtimeNotifier	CommunityRealtimeNotifier	SignalR CommunityHub
IChatPresenceQuery	PresenceTracker	(in-memory)
IAuditService	AuditService	—
IUserAdministration	UserAdministration	—
IAdminReadService	AdminReadService	—
IConsultantReadService	ConsultantReadService	—
IChatContactReadService	ChatContactReadService	—
ICurrentUserService	CurrentUserService	(HttpContext)
IDateTimeService	DateTimeService	—
<i>13 job ports</i>	<i>13 job classes (Infrastructure/Jobs)</i>	Hangfire (when enabled)

Adapters shown in *italics* are the development / configuration fall-backs (*Stub**, *NoOp**, *Local**) selected when the corresponding cloud service is not configured. The AI and embedding ports each have *three* adapters: Azure OpenAI, OpenAI-direct, and an offline local implementation, chosen by the `Ai:Provider` setting.